

12-系统设计

系统设计面试题

1. 设计方法论

问：系统设计面试怎么答？

答：

1. 需求澄清（功能需求、非功能需求、规模估算）
2. 容量估算（QPS、存储、带宽）
3. 高层设计（画架构图）
4. 详细设计（核心模块、数据模型、API）
5. 扩展与优化（瓶颈分析、权衡）

估算参考：– 日活 1000 万 → QPS 约 1000~10000 – 1 条推文 300 字节 → 1 亿条 ≈ 30GB

2. 经典题目

问：如何设计短链服务？

答：

功能：长 URL → 短 URL，短 URL 重定向到长 URL

方案：1. **短链生成**：自增 ID + Base62 编码 / MurmurHash 截断 2. **存储**：MySQL (id, short_url, long_url, create_time) + Redis 缓存热点 3. **重定向**：301（永久缓存）或 302（可统计点击量） 4. **过期清理**：定时任务删除过期短链

优化：– 布隆过滤器防穿透 – 分库分表（按 short_url hash） – CDN 缓存 301

问：如何设计秒杀系统？

答：

挑战：高并发、超卖、恶意请求

方案：

前端：按钮置灰、验证码
网关：限流、黑名单
服务层：
1. Redis 预减库存 (DECR, 库存 < 0 拒绝)
2. 异步 MQ 下单 (削峰)
3. 数据库乐观锁 (version 字段) 防超卖
4. 令牌桶限流

注意：– Redis 和 DB 库存一致性 – 消息幂等 – 热点 key 问题

问：如何设计 Feed 流（朋友圈/微博）？

答：

模式	做法	优缺点
推模式（写扩散）	发帖时写入所有粉丝收件箱	读快，大 V 发帖慢
拉模式（读扩散）	读时聚合关注的人帖子	写快，读慢
推拉结合	普通用户推，大 V 拉	业界主流

存储：– 发帖表：post_id, user_id, content, time – 收件箱 (Redis ZSet)：user_id → {post_id: timestamp} – 大 V 粉丝过多时走拉模式

问：如何设计定时任务系统？

答：

方案：1. 时间轮 (Netty HashedWheelTimer)：O(1) 插入 2. 延迟队列 (Redis ZSet / RocketMQ 延迟消息) 3. 分布式锁 + 数据库扫表

架构：

调度中心 → 分配任务到执行器 → 执行器执行 → 回调结果

注意：任务幂等、失败重试、分片广播。

问：如何设计点赞系统？

答：特点：高频读写，需返回「总赞数」+「当前用户是否已赞」。

Redis 方案：

Set article:likes:{postId} → 存点赞 userId, sismember 判是否已赞, scard 得总数
或 String + Set 分离：INCR 维护总数，Set 维护状态

持久化：Redis 成功后发 MQ → 消费者批量写 MySQL（异步削峰）。

热点 Key：大 V 内容用本地缓存 (Caffeine) 合并增量，减少 Redis 压力。

问：如何设计附近的人？

答：1. GeoHash：将二维坐标编码为一维字符串，前缀相同则距离近 2. Redis GEO：GEOADD、GEORADIUS 3. 方案：用户上报位置 → Redis GEO 存储 → 查询时 GEORADIUS

优化： – 按 GeoHash 前缀分片 – 定期清理不活跃用户

问：如何设计限流系统？

答：

请求 → 网关限流（令牌桶）→ 服务限流（Sentinel）→ 业务处理

维度： – 全局限流 – 接口级限流 – 用户级限流 – IP 级限流

算法：令牌桶（允许突发）、滑动窗口（平滑）

3. 高可用

问：如何设计高可用系统？

答：

手段	说明
冗余	多副本、多机房
负载均衡	Nginx、LVS、DNS
熔断降级	Sentinel，快速失败
限流	保护系统不被压垮
超时重试	合理超时 + 有限重试
异地多活	多机房同时服务

问：熔断器三种状态？

答：

Closed（正常）→ 错误率超阈值 → Open（熔断，快速失败）
↓ 超时报
Half-Open（试探请求）
↓ 成功 → Closed
↓ 失败 → Open

4. 数据库设计

问：分库分表策略？

答：

方式	说明
垂直分库	按业务拆库（用户库、订单库）
垂直分表	大字段拆出（如文章内容）
水平分库	同表数据分散到多个库
水平分表	同库同表拆为多个表

分片键选择：用户 ID、订单 ID（高频查询字段）

问题：跨库 JOIN、分布式事务、全局 ID、扩容迁移。

中间件：ShardingSphere、MyCat。

问：缓存和数据库一致性？

答：

策略	说明
Cache Aside	读：先缓存后 DB；写：先更新 DB 再删缓存
Read/Write Through	缓存层代理读写
Write Behind	先写缓存，异步写 DB

推荐：先更新 DB，再删除缓存（延迟双删）。

5. API 设计

问：RESTful API 设计原则？

答：– 资源用名词： `/users/{id}` – HTTP 方法表语义：GET 查、POST 增、PUT 改、DELETE 删 – 状态码规范：2xx 成功、4xx 客户端错误、5xx 服务端错误 – 版本控制： `/api/v1/users` – 分页： `?page=1&size=20`

6. 短链服务深入

问：短链服务详细设计方案？

答：

API 设计:

```
POST /api/v1/shorten { "longUrl": "...", "expireDays": 30 } → { "shortUrl": "https://t.cn/abc123" }
GET  /{shortCode}    → 302 Redirect
GET  /api/v1/stats/{shortCode} → 点击统计
```

短码生成:

方案	原理	优缺点
自增 ID + Base62	DB 自增 → 转 62 进制 (a-zA-Z0-9)	简单有序, 需分布式 ID
MurmurHash + 截断	长 URL 哈希取前 7 位	可能冲突, 需重试
预生成池	离线批量生成短码入池	高性能, 需维护池

存储:

```
CREATE TABLE url_map (
  id BIGINT PRIMARY KEY,
  short_code VARCHAR(10) UNIQUE,
  long_url VARCHAR(2048),
  create_time DATETIME,
  expire_time DATETIME,
  click_count BIGINT DEFAULT 0
);
```

读优化: Redis GET short:{code} → Miss 查 DB → 回写缓存。

问: 短链哈希冲突如何处理?

答:

- 1. 生成短码后先查 DB/Redis 是否存在
- 2. 冲突则: 自增 ID 方案不会冲突; Hash 方案则加盐重新哈希或取更长位数
- 3. 布隆过滤器: 快速判断"一定不存在", 减少 DB 查询
- 4. 短码长度: 6 位 Base62 ≈ 568 亿, 7 位 ≈ 3.5 万亿

MurmurHash 冲突: 生日悖论, 约 $\sqrt{N}$ 个 URL 后出现碰撞, 7 位足够亿级。

7. 秒杀系统深入

问: 秒杀系统完整方案?

答:

架构分层:

CDN 静态页 → 网关限流 → 秒杀服务 → Redis 预减库存 → MQ 异步下单 → 订单服务 → DB

前端：- 活动页静态化 CDN - 按钮置灰 + 倒计时 - 验证码/答题防刷

网关：- 令牌桶限流（如 1 万 QPS）- 用户黑名单、IP 限流 - 登录校验

服务层：

1. 活动开始前：库存预热到 Redis (DECR 原子操作)
2. 请求到达：校验活动状态、用户资格（一人一单）
3. Redis DECR 成功 → 发 MQ 创建订单
4. 消费者：DB 乐观锁 UPDATE stock SET count=count-1 WHERE id=? AND count>0 AND version=?
5. 失败则回滚 Redis 库存（或异步对账）

防超卖：Redis 预减 + DB 乐观锁/悲观锁 + 订单幂等。

热点 Key：Redis Cluster 本地缓存 + 请求合并。

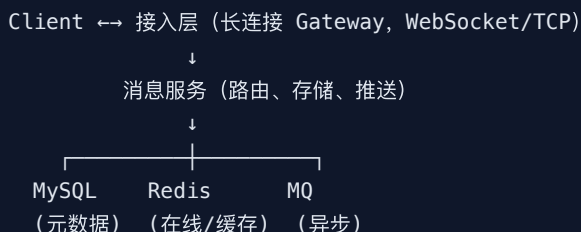
8. IM 系统设计

问：如何设计 IM（即时通讯）系统？

答：

功能：单聊、群聊、消息已读、离线消息、推送、文件传输。

架构：



消息流程：

1. A 发消息 → Gateway A → 消息服务
2. 消息服务：持久化 + 查 B 是否在线
3. B 在线 → Gateway B 推送；B 离线 → 存离线表/推送通知
4. 群聊：写扩散（每人收件箱）或读扩散（群消息表 + 拉取）

协议：WebSocket 全双工；Protobuf 序列化减小体积。

未读数：Redis INCR unread:{userId}:{convId}，已读时清零。

9. 排行榜设计

问：如何设计实时排行榜？

答：

Redis ZSet：

```
ZADD leaderboard:{gameId} score userId
ZREVRANGE leaderboard:{gameId} 0 9 WITHSCORES    # Top 10
ZRANK leaderboard:{gameId} userId                # 我的排名
```

大数据量：- 分片：按 score 范围或 userId hash 分多个 ZSet - 定时合并 Top N - 只维护 Top 1000，其余不精确排名

时间维度：日榜/周榜用不同 key + 定时过期。

防刷：限频加分、异常检测、人工审核。

10. 容量估算

问：系统设计中的容量估算公式？

答：

QPS 估算：

```
日活 DAU = 1000 万
平均每用户每天请求 50 次
平均 QPS = DAU × 50 / 86400 ≈ 5800
峰值 QPS = 平均 × 3~5 ≈ 2~3 万
```

存储估算：

```
每条记录大小 × 日增量 × 保留天数 × 副本数
例：短链 500 字节 × 100 万/天 × 365 天 × 3 副本 ≈ 500GB
```

带宽估算：

```
峰值 QPS × 平均响应大小
例：1 万 QPS × 10KB = 100MB/s ≈ 800Mbps
```

内存估算：

```
热点数据比例 × 总数据量
例：20% 热点 × 500GB = 100GB → 需 Redis 集群
```

服务器估算：

峰值 QPS / 单机能承受的 QPS (压测得出, 如 500)
例: $30000 / 500 = 60$ 台, 考虑冗余 $\times 1.5$

问: 如何设计微博/朋友圈 Feed 流?

答: (补充推拉细节)

推模式 (写扩散):

发帖 $\rightarrow$ 写入自己的发件箱 $\rightarrow$ 异步推送到所有粉丝的收件箱 (Redis ZSet)
读 $\rightarrow$ 直接读自己的收件箱
适合: 粉丝少的普通用户

拉模式 (读扩散):

发帖 $\rightarrow$ 只写入发件箱
读 $\rightarrow$ 拉取关注列表所有人的发件箱, 归并排序
适合: 粉丝百万的大 V

推拉结合: - 粉丝 < 5000 : 推 - 粉丝 ≥ 5000 : 拉 - 活跃粉丝推, 非活跃拉

11. 其他经典设计

问: 如何设计分布式 ID 发号器?

答:

方案: 号段模式 (Leaf) 或雪花
API: GET /api/v1/id/next $\rightarrow$ 123456789
要求: 全局唯一、趋势递增、高性能、高可用
部署: 多实例 + DB 号段 / 雪花 workerId 分配 (ZK)

问: 如何设计接口限流系统?

答: (补充实现细节)

架构:
Client $\rightarrow$ API Gateway (全局限流) $\rightarrow$ 服务 (接口级限流) $\rightarrow$ 业务
限流维度: IP、用户 ID、接口、租户
存储: Redis 集中计数 (分布式) 或本地令牌桶 (单实例)
响应: 429 Too Many Requests + Retry-After 头

12. 高频设计题

问：如何设计 RPC 框架？

答：目标：调用远程服务像调用本地方法。

核心模块：

- 1. 动态代理：拦截接口调用，收集类名/方法/参数
- 2. 序列化：Protobuf/Hessian → 字节流
- 3. 自定义协议：长度 + 魔数 + 序列化类型（解决 TCP 粘包）
- 4. 网络通信：Netty NIO
- 5. 注册中心：Nacos/ZK，服务注册与发现
- 6. 负载均衡：轮询/随机/一致性 Hash
- 7. 容错：超时、重试、熔断（Failover/Failfast）
- 8. SPI 插件化：序列化/负载均衡可扩展

问：订单 30 分钟未支付如何自动取消？

答：

方案	原理	优缺点
定时扫表	Scheduled 扫超时订单	简单，DB 压力大
DelayQueue	JDK 延迟队列	内存占用，无持久化
时间轮	Netty HashedWheelTimer	大量定时任务高效
Redis ZSet	score=过期时间戳，定时 ZRANGEBYSCORE	分布式友好
Redis Key 过期	监听 key 过期事件	不精确，可能丢事件
MQ 延迟消息	RocketMQ/RabbitMQ 延迟投递	大厂主流，异步可靠
死信队列	TTL 过期进 DLQ	精度略差

推荐：下单成功发 30min 延迟消息 → 消费时校验状态 → 取消订单 + 归还库存（Redis + DB）。

问：系统 QPS 突然提升 10 倍怎么应对？

答：阶段一（紧急止血）： – 网关限流（Sentinel），超阈值快速失败 – 业务降级：关闭非核心（推荐、评论、积分） – K8s/云弹性扩容

阶段二（架构演进）：

- 读：本地缓存（Caffeine）+ Redis + MySQL 多级缓存
- 写：MQ 异步削峰，按 DB 真实能力消费
- 库：读写分离 + 分库分表

问：单 Redis 无法承压大流量怎么办？

答：

方案	说明
读写分离	Master 写，多 Slave 读
Redis Cluster	16384 slot 分片，水平扩展
本地缓存	热点数据进 JVM，减少 Redis 访问
拆分 Key	热点 Key 加随机后缀分散到多 slot
多级缓存	CDN → 本地 → Redis → DB

问：如何设计可重入分布式锁？

答：可重入：同一线程多次加锁，计数 +1；释放时 -1，为 0 才真正释放。

Redis 实现（Hash 结构）：

```
Key: lock:resource
Field: threadId Value: count
加锁: HINCRBY + EXPIRE; 同 threadId 则 count++
释放: HINCRBY -1; count=0 则 DEL
```

Redisson：RLock 原生支持可重入 + 看门狗续期。

ZK：同一会话可重复获取同一节点锁。

问：负载均衡有哪些方案？

答：

类型	方案	特点
硬件	F5 等	性能强，成本高
软件	Nginx、LVS、Gateway	灵活，主流
DNS	多 A 记录	简单，无法感知服务器状态
CDN	边缘节点	静态资源，降低延迟

算法：轮询、加权轮询、随机、IP Hash、一致性 Hash、最少连接。

问：微服务架构是什么？和单体怎么选？

答：微服务：按业务边界拆成独立服务，独立开发/部署/扩展。

组件：注册中心（Nacos）、RPC/REST、网关、配置中心、链路追踪（SkyWalking）、熔断（Sentinel）。

优点：灵活扩展、团队并行、技术栈异构。 缺点：分布式复杂度、运维成本、分布式事务。

选型： 小团队/简单业务用单体；业务复杂、团队大、需快速迭代用微服务。

问：单体项目如何拆分成微服务？

答：

1. 梳理模块依赖，画依赖图
2. 按 DDD 限界上下文划分（用户/订单/商品/支付）
3. 先拆耦合弱的模块（如通知、积分）
4. 核心链路（下单）最后拆
5. 数据库按服务拆分，禁止跨库 JOIN
6. 引入网关、注册中心、配置中心
7. 灰度迁移，双写/对账保证数据一致

原则： 渐进式拆分，避免大爆炸式重写。

问：对外 API 超时如何排查？

答：

1. 确认超时范围：全量 vs 特定接口/用户/地域
 2. 监控：APM (SkyWalking/Prometheus) 看 P99 延迟
 3. 链路追踪：TraceID 定位慢在哪个服务/Span
 4. 分层排查：
 - 网关：限流、连接池满
 - 服务：GC、线程池满、慢 SQL
 - 中间件：Redis/MySQL/MQ 延迟
 - 网络：DNS、跨机房 RTT
 5. 日志：错误栈、慢查询日志
 6. 压测复现，对比正常基线
-

图解加深

专题	设计要点
短链	Base62、302 vs 301、布隆过滤器
秒杀	分层防御、Redis 预减、MQ 削峰
Feed 流	推拉结合、收件箱 ZSet
IM	长连接、消息路由、离线推送
排行榜	ZSet、分片、定时榜
容量估算	QPS、存储、带宽、机器数公式
RPC 框架	代理、序列化、Netty、注册中心
点赞系统	Redis Set、MQ 异步落库
订单超时	延迟消息、Redis ZSet
微服务	拆分原则、注册/网关/追踪

大厂追问

1. **短链用 301 还是 302?** 301 浏览器永久缓存，无法统计点击量；302 每次走服务端，可统计，SEO 权重不转移。
2. **秒杀库存 Redis 和 DB 不一致怎么办?** 定时对账、以 DB 为准；或最终一致性，MQ 消费失败回滚 Redis。
3. **IM 消息如何保证不丢?** 客户端 ACK、服务端持久化、离线消息表、重连后拉取增量。
4. **10 亿用户排行榜怎么存?** 不可能全量 ZSet，只维护 Top 10 万 + 分片 + 近似排名。
5. **设计题如何开场?** 先澄清功能/非功能需求、DAU/QPS，再画架构，最后深入 1~2 个模块。
6. **如何做读写分离下的缓存一致性?** 先更新 DB 再删缓存；延迟双删；订阅 Binlog 异步删缓存。
7. **10 万人抢 10 个库存怎么设计?** 入口限流 → Redis Lua 原子扣减 + 一人一单 → MQ 异步下单 → DB 乐观锁兜底 → 订单超时释放库存 + 幂等。
8. **点赞为什么用 Set 不用 List?** Set $O(1)$ 判重 (sismember)，List 判重 $O(n)$ ；Set scard 得总数。
9. **RPC 为什么用自定义协议?** 解决 TCP 粘包/半包，携带魔数、长度、序列化类型，方便拆包。